

Advanced CSS. Flexbox & Grid.

Assignment 2 · AKYL GROUP

Student: Jomart Akylbek
Group: IT2507
Date: 27 September 2026

Project overview

The assignment extends the existing real-estate website with three exercises: a geometric composition, a component library and an editorial page with three layouts. The new pages share a green palette, typography and spacing.

The implementation uses HTML and external CSS. There is no JavaScript, Bootstrap or media query in the assignment pages. Layouts adapt using fluid sizing, wrapping and intrinsic content sizes.

Source code

<https://github.com/akylzhomart699-spec/AKYL-GROUP>

Assignment directory: assignment2/

Contents

1. Geometric composition and key CSS
2. Five Flexbox components and key CSS
3. Three layouts with identical markup and key CSS
4. Responsive checks and conclusion

One grid, seven blocks.

A single Grid container has six equal columns and six equal rows. Each colored block occupies the cells required by the supplied drawing.

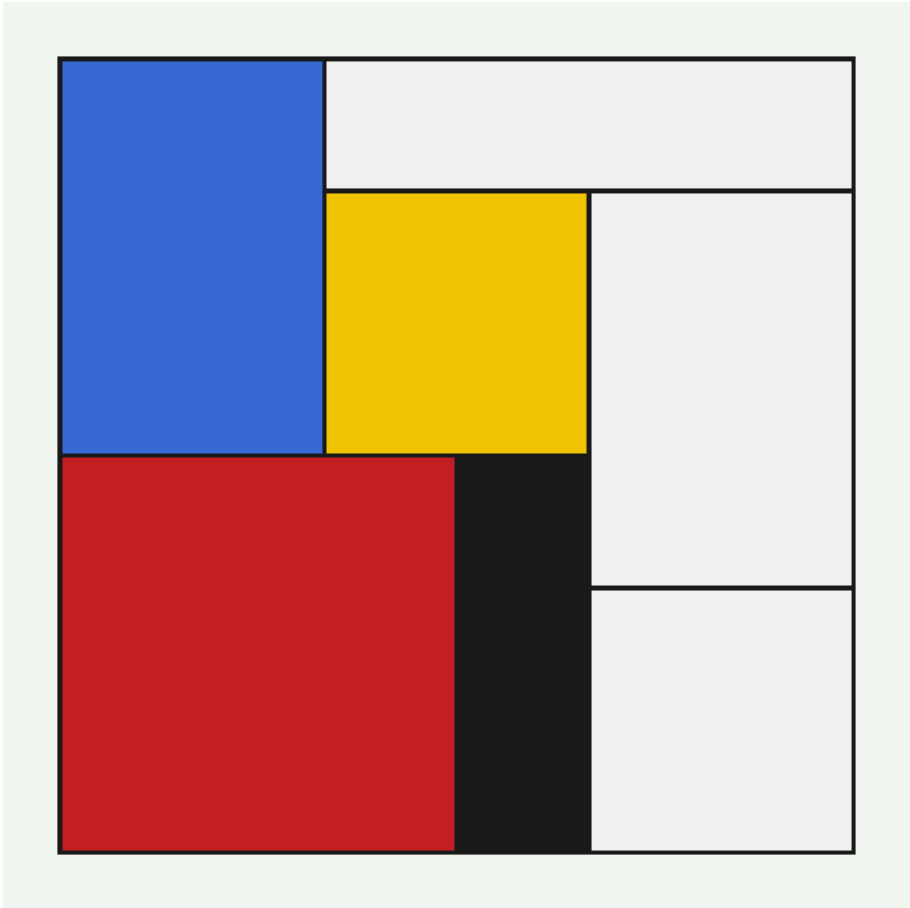


Figure 1. Browser screenshot of the implemented composition.

Block	Rows occupied	Columns occupied
Blue	1–3	1–2
Top white	1	3–6
Yellow	2–3	3–4
Right white	2–4	5–6
Red	4–6	1–3
Black	4–6	4
Bottom white	5–6	5–6

Proportions that scale.

key-composition · key declarations (visual styling omitted)

```
.composition {  
  --gap: .5vmin;  
  display: grid;  
  grid-template-columns: repeat(6, 1fr);  
  grid-template-rows: repeat(6, 1fr);  
  gap: var(--gap);  
  padding: var(--gap);  
  inline-size: min(100%, 88vmin);  
  aspect-ratio: 1;  
  margin-inline: auto;  
  background: #1c1c1c;  
}  
  
.blue      { grid-area: 1 / 1 / 4 / 3; }  
.top-white { grid-area: 1 / 3 / 2 / 7; }  
.yellow    { grid-area: 2 / 3 / 4 / 5; }  
.right-white { grid-area: 2 / 5 / 5 / 7; }  
.red       { grid-area: 4 / 1 / 7 / 4; }  
.black     { grid-area: 4 / 4 / 7 / 5; }  
.bottom-white { grid-area: 5 / 5 / 7 / 7; }
```

Figure 2. Key declarations from composition.css; block colors omitted.

`aspect-ratio: 1` keeps the composition square. Its width is limited to the smaller of the available space and `88vmin`, so the square cannot become taller than the viewport.

The black container background is visible through the Grid gaps. The blocks do not use borders or margins to create internal lines. Matching container padding creates the outside frame.

The four numbers in `grid-area` specify row start, column start, row end and column end. End lines are exclusive: the blue block ends at row line 4 and therefore covers three rows.

Navigation and cards.

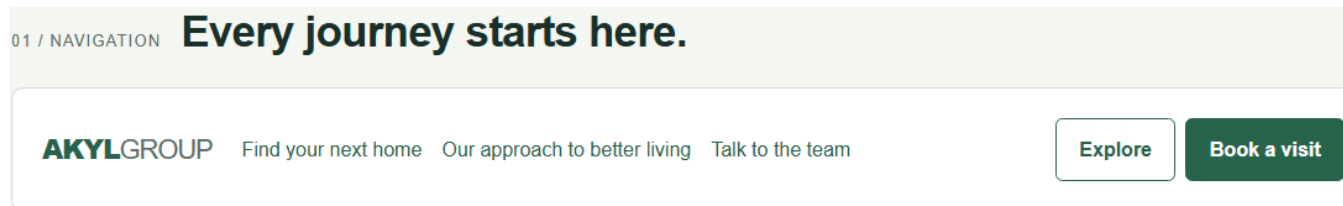


Figure 3. A logo, long links and a right-aligned action group.

The navigation is a wrapping Flexbox row. Links can grow, while an automatic start margin moves the action group to the right edge, including when it wraps.

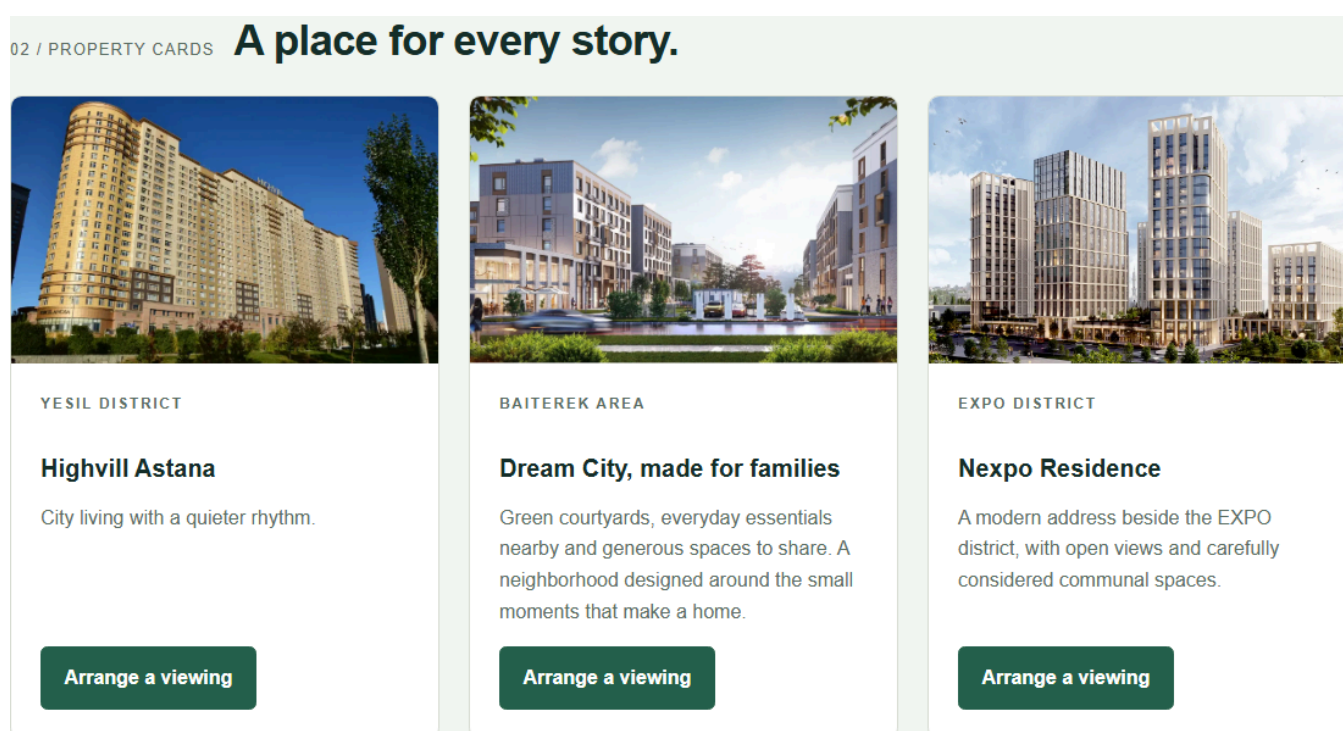


Figure 4. Different description lengths, equal card heights and aligned buttons.

Cards stretch to the height of the tallest card in each row. Each card body is a growing vertical Flexbox container. The button's automatic top margin consumes unused space, so all buttons align without a fixed card height.

Small details, flexible layouts.



Figure 5. Centered page links and a right-aligned result count.

Equal flexible regions on both sides center the page controls. The result count occupies the right region; the parts may wrap on narrow screens.

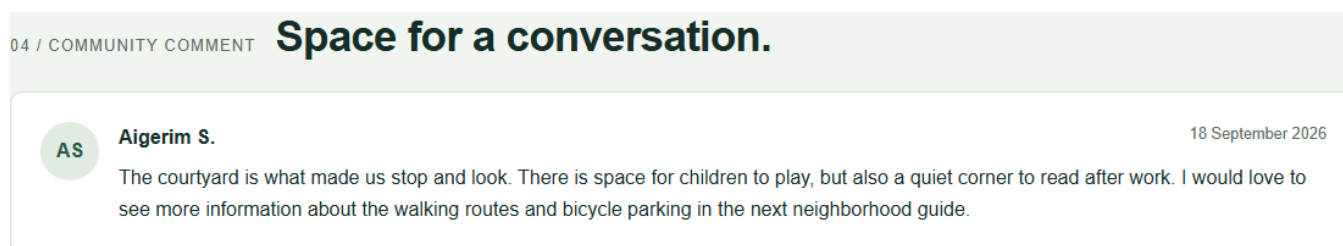


Figure 6. Text remains beside the avatar.

The avatar does not shrink. The text has its own flexible child with a zero minimum inline size and long-word wrapping, so it never flows underneath the avatar.

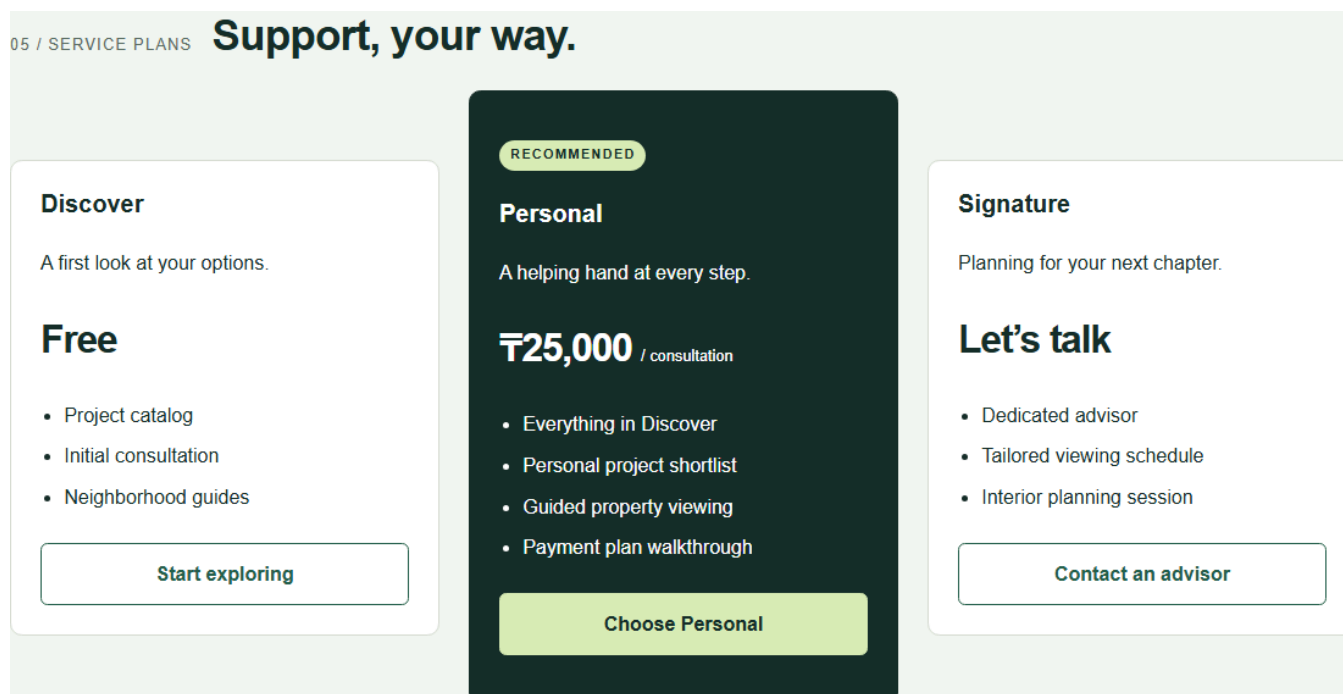


Figure 7. The recommended plan is naturally taller than its neighbors.

The pricing row vertically centers its cards. The middle card has an extra badge, feature and vertical padding; its height comes from content rather than a fixed value.

Wrapping and alignment.

key-components-a · key declarations (visual styling omitted)

```
.demo-nav {  
  display: flex;  
  flex-wrap: wrap;  
  align-items: center;  
}  
.nav-links { display: flex; flex: 1 1 20em; flex-wrap: wrap; }  
.nav-actions { display: flex; margin-inline-start: auto; }  
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  align-items: stretch;  
}  
.property-card {  
  display: flex;  
  flex-direction: column;  
  flex: 1 1 17rem;  
  min-inline-size: 0;  
}  
.card-content { display: flex; flex-direction: column; flex: 1; }  
.card-content .button { margin-block-start: auto; }
```

Figure 8. Layout declarations from components.css.

`flex: 1 1 17rem` is a flexible preferred size: cards may grow or shrink. It is not a fixed width. `align-items: stretch` equalizes heights within each row.

All five components share `shared.css` for design tokens and `components.css` for their layout rules. Grid, floats and fixed layout widths or heights are not used in these components. Only the avatar has a fixed size.

Let content set the size.

key-components-b · key declarations (visual styling omitted)

```
.pagination { display: flex; flex-wrap: wrap; }
.pagination::before { content: ""; flex: 1 1 8rem; }
.page-numbers {
  display: flex;
  flex: 0 1 auto;
  flex-wrap: wrap;
  justify-content: center;
}
.result-count { flex: 1 1 8rem; text-align: end; }
.comment { display: flex; align-items: flex-start; }
.avatar { flex: 0 0 3rem; inline-size: 3rem; block-size: 3rem; }
.comment-body {
  flex: 1;
  min-inline-size: 0;
  overflow-wrap: anywhere;
}
.pricing { display: flex; flex-wrap: wrap; align-items: center; }
.plan { display: flex; flex: 1 1 17rem; flex-direction: column; }
.plan.recommended { padding-block: 2.5rem; }
```

Figure 9. The remaining Flexbox layout declarations.

The pagination's empty pseudo-element balances the result count on wide screens. The comment uses separate avatar and text flex items. Pricing uses natural heights and center alignment.

Shared spacing variables keep gaps and padding consistent. Colors, fonts and button styles are reused rather than restyled independently for each exercise.

Twelve stories, one markup.

The page contains twelve articles with different title and description lengths. Each article contains an image, a text body and a date.

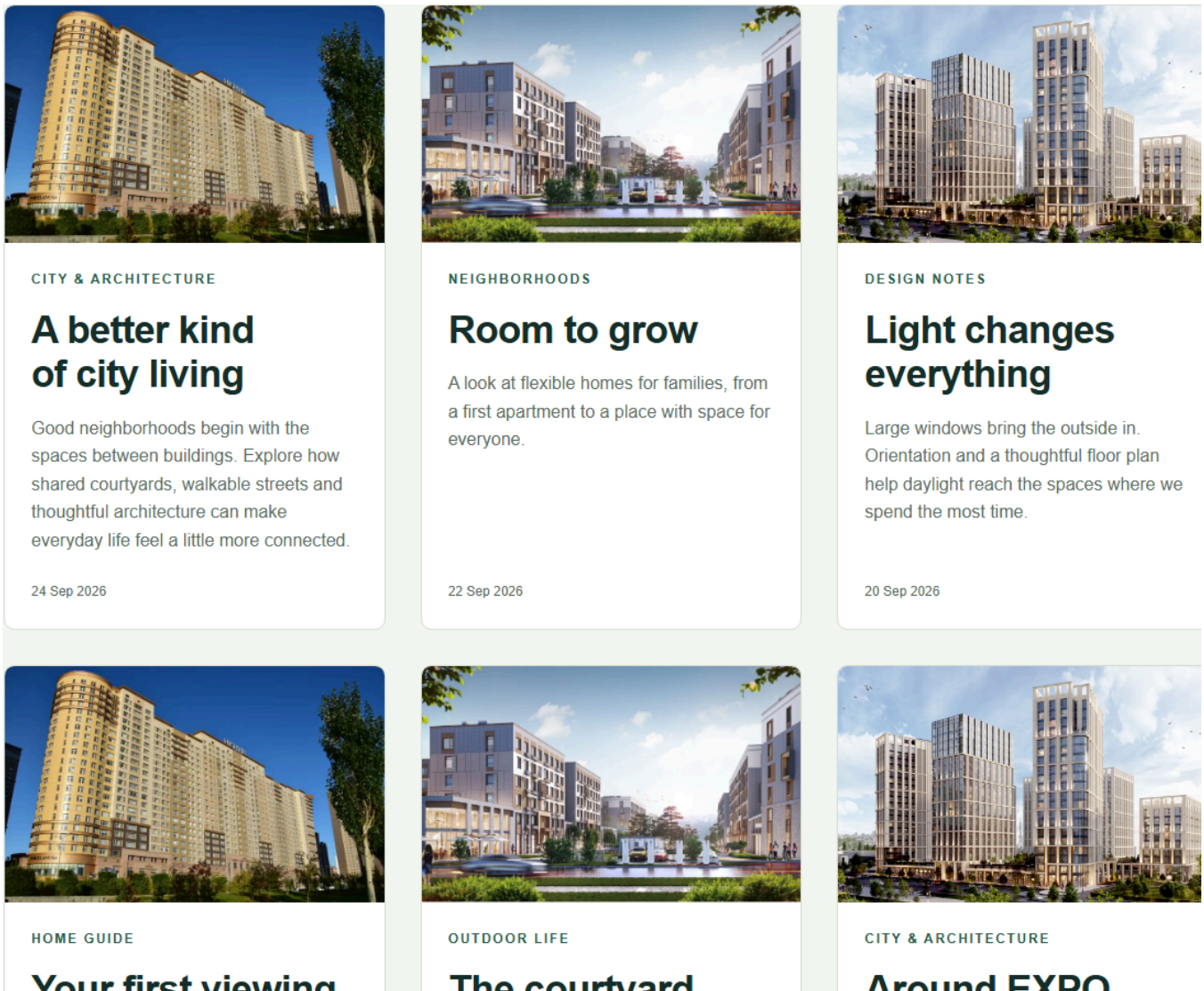


Figure 10. Upper portion of the card-grid view at a 1200 px viewport.

Tool: Grid uses `repeat(auto-fit, minmax(min(100%, 19rem), 1fr))` to choose the column count, while Flexbox organizes each card's content vertically.

The inner `min()` allows a track to fit a narrow container instead of forcing horizontal overflow.

The same stories, in rows.



CITY & ARCHITECTURE

A better kind of city living

24 Sep 2026

Good neighborhoods begin with the spaces between buildings. Explore how shared courtyards, walkable streets and thoughtful architecture can make everyday life feel a little more connected.



NEIGHBORHOODS

Room to grow

22 Sep 2026

A look at flexible homes for families, from a first apartment to a place with space for everyone.



DESIGN NOTES

Light changes everything

20 Sep 2026

Large windows bring the outside in. Orientation and a thoughtful floor plan help daylight reach the spaces where we spend the most time.

Figure 11. Upper portion of the compact-list view at a 1200 px viewport.

Tool: A single-column Grid stacks the articles; inside each item, a horizontal Flexbox row places the image left, text in the middle and date at the right edge.

The image uses a proportional width. The body can shrink and wrap, while the date keeps a small readable area. No changes to the article HTML are needed.

A lead story sets the rhythm.

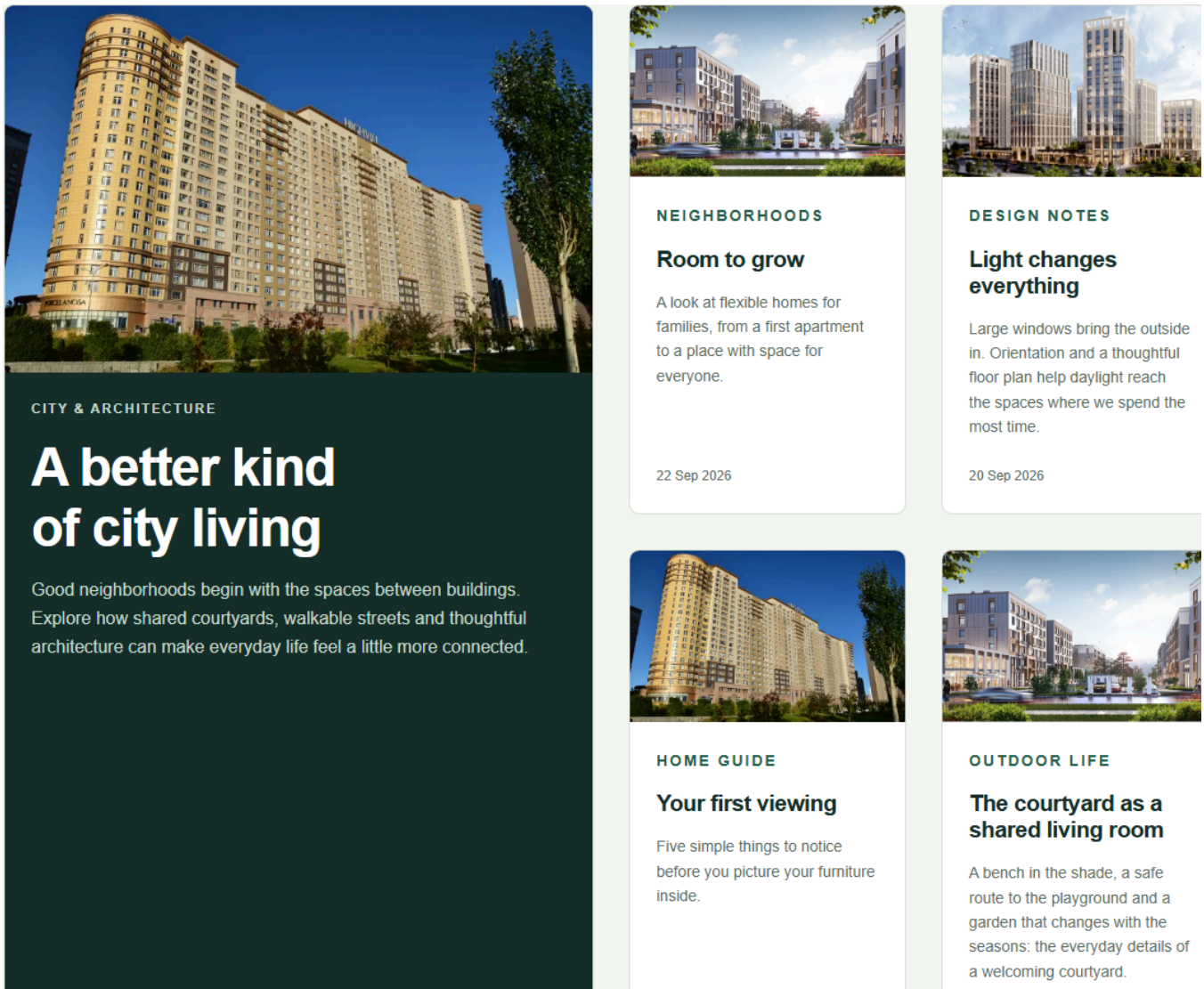


Figure 12. Upper portion of the magazine view at a 1200 px viewport.

Tool: Grid lets the first item span two columns and two rows, while Flexbox structures the content inside each article.

The track minimum can shrink to half the available width. On narrow screens the lead occupies both columns; on wide screens additional columns appear and smaller stories fill the space beside it.

Change a class, keep the HTML.

key-layouts · key declarations (visual styling omitted)

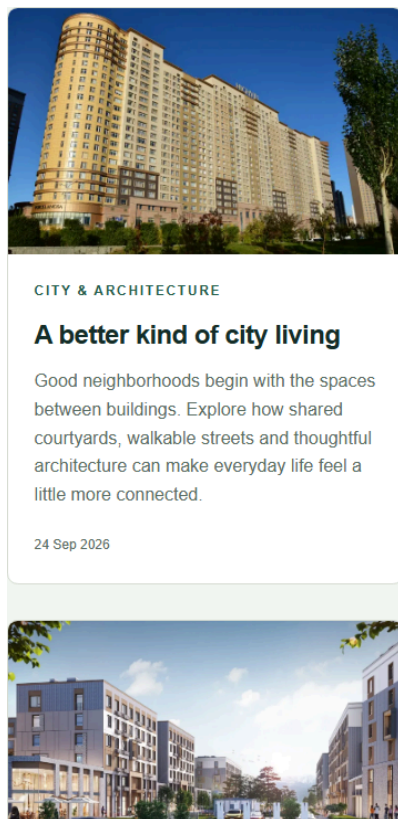
```
.journal { display: grid; gap: var(--space-4); }
.journal-item { display: flex; flex-direction: column; }
.story-body { display: flex; flex-direction: column; flex: 1; }
.layout-grid {
  grid-template-columns:
    repeat(auto-fit, minmax(min(100%, 19rem), 1fr));
}
.layout-list { grid-template-columns: minmax(0, 1fr); }
.layout-list .journal-item {
  flex-direction: row;
  align-items: center;
}
.layout-list .journal-item > img { flex: 0 0 21%; }
.layout-list .journal-item > time { margin-inline-start: auto; }
.layout-magazine {
  grid-template-columns: repeat(auto-fit,
    minmax(min(calc((100% - var(--space-4)) / 2), 14rem), 1fr));
  grid-auto-flow: row dense;
}
.layout-magazine .journal-item:first-child {
  grid-column: span 2;
  grid-row: span 2;
}
```

Figure 13. Key declarations from layouts.css.

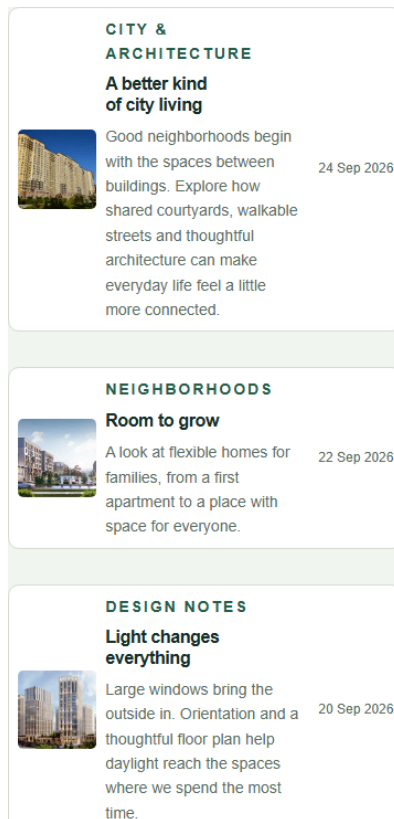
Change the container class from `journal layout-grid` to `journal layout-list` or `journal layout-magazine`. All twelve articles remain untouched.

Three static demonstration files make each mode accessible without JavaScript. A file comparison confirmed they are identical except for this single class value.

From desktop to phone.



Card grid, 390 px.



Compact list, 390 px.



Magazine, 390 px.

These screenshots show the upper portion of each layout at a narrow viewport. The same rules are used at every width; no media queries are needed.

Browser check	Result
All five pages at 1440, 390 and 320 px	No horizontal overflow
Images on every page	All loaded
Composition dimensions	Square; within viewport height
Cards at desktop width	Equal heights; aligned buttons
Recommended pricing plan	Taller than adjacent plans
Articles in every layout	12; identical markup

Conclusion

The exercises show how Grid handles two-dimensional placement and how Flexbox handles alignment and distribution along one axis. Fluid track sizes, wrapping, automatic margins and content-driven heights make layouts respond to both screen size and changing text.